Advanced Algorithms

Programming Assignment 2

Individual Reflection

Elan Blaustein – 24958098

November 4, 2026

Worst Case

Across all the problem instance datasets used in the experiment, the one that consistently performed the worst was the Falkenauer_T set. The reason it was the hardest set of problems, despite not being particularly large, comes down to the intrinsic problem structure. This problem set is based off of the 3-partition problem. As explained in the main report, the 3-partition is a Strongly NP-complete problem that is polynomially reducible to Bin Packing, where all items must be packed perfectly into equal groups of three. The Falkenauer_T set is comprised of instances of 3-partition problems reduced to BP problems. This suggests that this problem set has an intrinsic NP-hardness to find an optimal solution, as compared to other problem sets in our library, such as Randomly_Generated. It is specifically these tight-bound, NP-hard problem structures with a single, difficult-to-obtain solution that creates the biggest problem for my approximate algorithm.

Implementation Decision

I didn't have a particularly momentous bug/design decision during the implementation of the FFD and MFFD algorithms. I have gone through a long laundry list of minor bugs and errors (clearly seen in the repository commit log), but the simplicity of the FFD structure made implementation very straightforward. However, the most notable issue I had for FFD was during the initial construction of the algorithm; initial testing logs returned approximate solutions way larger than the $\frac{11}{9}$ approximation factor. It didn't take me particularly long to find the root cause, I had accidentally pre-sorted the item list in the wrong direction, forgetting to include `std::greater`, but I found it somewhat notable that it took me a while to notice there was even an issue in the first place. What the program was essentially doing was a worst-case (non-decreasing) First-Fit algorithm, which still had a guaranteed approximation factor of $\frac{17}{10}$.

I did have some issues during the implementation of MFFD; my insistence on using multiset iterators to optimise the data storage/manipulation caused me plenty of segmentation faults. I had to be cautious with `.end()` and `.rbegin()` in conjunction with `std::prev()` and `std::next()`, manually moving about the item classes (a big part of frustration was forgetting `.end()` does not point to the last element). Overall though, I had no overarching design issue beyond memory manipulation, and I was able to put together the algorithms in a satisfyingly optimised manner.

Design Changes

Due to the simplicity of the FFD algorithm, there isn't much that I would (or could) really do to affect it. Perhaps there are certain elements of the MFFD algorithm that could be further optimised with regards to memory management, but the algorithms seem to work perfectly fine, entirely within the expected bounds. If anything, one fundamental aspect that I would change would be to instead implement the assignment algorithm, than just the optimisation. That is to say, the report only focused on finding the optimal number of bins, as that is how BP problems are typically benchmarked. However, I have some interest in the alternate methodology where we implemented algorithms that returned the exact item packing assignment, and then analysed them to see how exactly the different algorithms performed the packing and if there were some observable patterns characteristic to each process. However, that felt a bit too beyond-scope, so the idea never came to fruition, but it was an interesting thought for an alternate world.

Algorithm Insight

I had summarised the majority of the theoretical proof behind the FFD algorithm's approximation boundary in the main report, but for transparency's sake, there were (and still are) many parts of the proof that I couldn't wrap my head around. The entire paper is essentially one long proof by contradiction, and as the implicating lemmas keep stacking higher, I have a harder time understanding the implicit deductions. For example, I simply could not figure out how the initial assumption of the proof implied Lemma 3, despite many attempts. However, the latter half of the proof, analysing the items with respect to size classes and forming a weighted sum that reaches the approximation bound, was much more tractable to me. It was specifically the implementation of the MFFD algorithm that got me to understand the role of the size classes and the utility within the proof (and made for a smooth segue in the report). For example, determining the Large size item class immediately creates a new lower bound $\text{OPT}(I) \geq |\text{Large}|$. I intend to keep trying to noodle out the proof in my own time, and I *will* figure out Lemma 3.